



Polymorphism

▼ Type	Lecture
📅 Date	@January 22, 2022
☰ Lecture #	1
🔗 Lecture URL	https://youtu.be/xvzxGV3pdMM
🔗 Notion URL	https://21f1003586.notion.site/Polymorphism-7f72e8aba0ab4f45ab160c5c3f6ad48a
# Week #	5

Polymorphism

- In object-oriented programming, polymorphism (*inheritance*) usually refers to the effect of dynamic dispatch
 - `S` is a sub-class of `T`
 - `S` overrides a method `f()` defined in `T`
 - Variable `v` of type `T` is assigned to an object of type `S`
 - `v.f()` uses the definition of `f()` from `S` rather than `T`
- Every object “knows” what it needs to do
- More generally, polymorphism refers to behaviour that depends only a specific capabilities — ***structural polymorphism***
 - Reverse an array / list (*should work for any type*)
 - Search for an element in an array / list (*need equality check*)
 - Sort an array / list (*need to compare values*)

Structural Polymorphism

- Uses the Java class hierarchy to simulate this
- Polymorphic `reverse`

```
public void reverse(Object[] arr) {
    int n = arr.length;

    for(int i = 0, i < n/2; ++i) {
        Object temp = arr[i];
        arr[i] = arr[(n - 1) - i];
        arr[(n - 1) - i] = temp;
    }
}
```

```
}  
}
```

- Polymorphic `find`
 - `==` translates to `Object.equals()`

```
public int find(Object[] arr, Object o) {  
    for(int i = 0; i < arr.length; ++i) {  
        if(arr[i] == o) return i;  
    }  
    return -1;  
}
```

- Polymorphic `sort`
 - Use interfaces to capture capabilities

```
public interface Comparable {  
    public abstract int cmp(Comparable s);  
}  
  
public class SortFunctions {  
    public static void quicksort(Comparable[] a) {  
        ...  
        // Usual code for quicksort, except that  
        // to compare a[i] and a[j] we use  
        // a[i].cmp(a[j])  
    }  
}
```

Type consistency

- Polymorphic function to copy an array
- Need to ensure that target array is type compatible with source array
 - Type errors should be flagged at compile time

```
public void arraycopy(Object[] src, Object[] target) {  
    int limit = Math.min(src.length, target.length);  
    for(int i = 0; i < limit; ++i) {  
        target[i] = src[i];  
    }  
}  
  
Date[] datearr = new Date[10];  
Employee[] emparr = new Employee[10];  
  
arraycopy(datearr, emparr); // Runtime error
```

- More generally source array can be a subtype of the target array
 - But the converse is illegal

```
public void arraycopy(Object[] src, Object[] target) {  
    int limit = Math.min(src.length, target.length);  
    for(int i = 0; i < limit; ++i) {  
        target[i] = src[i];  
    }  
}  
  
public class Ticket { ... }  
public class ETicket extends Ticket { ... }  
  
Ticket[] tktarr = new Ticket[10];  
ETicket[] etktarr = new ETicket[10];  
  
arraycopy(etktarr, tktarr); // Allowed  
// But arraycopy(tktarr, etktarr) is illegal
```

Polymorphic data structures

- Arrays, lists, ... should allow arbitrary elements

- A polymorphic list stores values of type `Object`

```
public class LinkedList {
    private int size;
    private Node first;

    public Object head() {
        Object returnval;
        ...
        return returnval;
    }

    public void insert(Object newdata) { ... }

    private class Node {
        private Object data;
        private Node next;
        ...
    }
}
```

- Two problems
 - Type information is lost, need casts

```
public class LinkedList {
    private int size;
    private Node first;

    public Object head() { ... }

    public void insert(Object newdata) { ... }

    private class Node { ... }
}

LinkedList list = new LinkedList();
Ticket t1, t2;

t1 = new Ticket();
list.insert(t1);
t2 = (Ticket)(list.head());
// head() returns an Object
```

- List need not be homogenous

```
public class LinkedList {
    private int size;
    private Node first;

    public Object head() { ... }

    public void insert(Object newdata) { ... }

    private class Node { ... }
}

LinkedList list = new LinkedList();
Ticket t = new Ticket();
Date d = new Date();
list.insert(t);
list.insert(d);
...
```

Generic Programming in Java

- Java added generic programming to address these issues
- Classes and functions can have type parameters
 - `class LinearList<T>` holds values of type `T`
 - `public T head() { ... }` returns a value of the same type `T` as enclosing class
- Can describe subclass relationships between type variables
 - `public <S extends T,T> void arraycopy(S[] src, T[] target) { ... }`



Generics programming in Java

Type	Lecture
Date	@January 23, 2022
Lecture #	2
Lecture URL	https://youtu.be/wqqOMQb0ft0
Notion URL	https://21f1003586.notion.site/Generics-programming-in-Java-c7282fcce5ca4faa844fcd0b12da63d5
Week #	5

Java Generics

- Use type variables
- Polymorphic `reverse` in Java
 - Type quantifier before return type
 - “For every type `T` ...”

```
public <T> void reverse(T[] arr) {
    T temp;
    int n = arr.length;

    for(int i = 0; i < n / 2; ++i) {
        temp = arr[i];
        arr[i] = arr[(n - 1) - i];
        arr[(n - 1) - i] = temp;
    }
}
```

- Polymorphic `find` in Java
 - Searching for a value of incompatible type is now a compile-time error

```
public <T> int find(T[] arr, T o) {
    for(int i = 0; i < arr.length; ++i) {
        if(arr[i] == o) return i;
    }

    return -1;
}
```

- Polymorphic `arraycopy`

- Source and target types must be identical

```
public <T> static void arraycopy(T[] src, T[] target) {
    int i, limit;
    limit = Math.min(src.length, target.length);

    for(i = 0; i < limit; ++i) {
        target[i] = src[i];
    }
}
```

- A more generous `arraycopy`
 - Source and target types may be different
 - Source type must extend target type

```
public <S extends T, T> static void arraycopy(T[] src, T[] target) {
    int i, limit;
    limit = Math.min(src.length, target.length);

    for(i = 0; i < limit; ++i) {
        target[i] = src[i];
    }
}
```

Polymorphic data structures

- A polymorphic list
- The type parameter `T` applies to the class as a whole
- Internally, the `T` in `Node` is the same `T`
- Also, the return value of `head()` and the argument of `insert()`

```
public class LinkedList<T> {
    private int size;
    private Node first;

    public T head() {
        T returnval;
        ...
        return returnval;
    }

    public void insert(T newdata) { ... }

    private class Node {
        private T data;
        private Node next;
        ...
    }
}
```

- Instantiate generic classes using concrete type

```
public class LinkedList<T> {
    ...
}

LinkedList<Ticket> ticketList = new LinkedList<Ticket>();
LinkedList<Date> dateList = new LinkedList<Date>();

Ticket t = new Ticket();
Date d = new Date();

ticketList.insert(t);
dateList.insert(d);
```

- Be careful not to accidentally hide a type variable


```
public <T> void insert(T newDate) { ... }
```
- `T` in the argument of `insert()` is a new `T`
- Quantifier `<T>` masks the type parameter `T` of `LinkedList`

- Contrast with

```
public <T> static void arraycopy(T[] src, T[] target) { ... }
```

```
public class LinkedList<T> {
    private int size;
    private Node first;

    public T head() {
        T returnval;
        ...
        return returnval;
    }

    public <T> void insert(T newdata) { ... }

    private class Node {
        private T data;
        private Node next;
        ...
    }
}
```

Summary

- Generics introduce structural polymorphism into Java through type variables
- Classes and functions can have type parameters
 - `class LinearList<T>` holds values of an arbitrary type `T`
 - `public T head() { ... }` returns a value of same type `T` used when creating the list
- Can describe subclass relationships between type variables
 - `public <S extends T, T> void arraycopy (S[] src, T[] target) { ... }`
- Be careful not to accidentally hide type variables

```
public <T> void insert(T newData) { ... } inside class LinkedList<T>
```

vs

```
public <T> static void arraycopy (T[] src, T[] target) { ... }
```



Java Generics and Subtyping

Type	Lecture
Date	@January 23, 2022
Lecture #	3
Lecture URL	https://youtu.be/Dd9rAJo5H9s
Notion URL	https://21f1003586.notion.site/Java-Generics-and-Subtyping-d795e88a149c4cb887aa1a25dc9a7c3c
Week #	5

Extending subtyping in contexts

- If `S` is compatible with `T`, `S[]` is compatible with `T[]`

```
ETicket[] elecarr = new ETicket[10];
Ticket[] ticketarr = elecarr;
// OK. ETicket[] is a subtype of Ticket[]
```

- But

```
...
ticketarr[5] = new Ticket();
// Not OK because ticketarr[5] refers to an ETicket
```

- A type error at run time
- Java array typing is covariant
 - If `S extends T` then `S[] extends T[]`

Generics and Subtypes

- Generic classes are not covariant
 - `LinkedList<String>` is not compatible with `LinkedList<Object>`
- The following will not work to print out an arbitrary `LinkedList`

```
public class LinkedList<T> { ... }

public static void printList(LinkedList<Object> l) {
    Object o;
```

```

    Iterator i = l.get_iterator();

    while(i.has_next()) {
        o = i.get_next();
        System.out.println(o);
    }
}

```

- How can we get around this limitation?

Generic methods

- As we have seen, we can make the method generic by introducing a type variable

```

public class LinkedList<T> { ... }

public static <T> void printList(LinkedList<T> l) {
    Object o;
    Iterator i = l.get_iterator();

    while(i.has_next()) {
        o = i.get_next();
        System.out.println(o);
    }
}

```

- `<T>` is a type quantifier: For every type `T`, ...
- Note that `T` is not actually used inside the function
 - We use `Object o` as a generic variable to cycle through the list

Wildcards

- Instead, use `?` as a wildcard type variable

```

public class LinkedList<T> { ... }

public static void printList(LinkedList<?> l) {
    Object o;
    Iterator i = l.get_iterator();

    while(i.has_next()) {
        o = i.get_next();
        System.out.println(o);
    }
}

```

- `?` stands for an arbitrary unknown type
- Avoids unnecessary type variable quantification when the type variable is not needed elsewhere
- We can define variables of a wildcard type

```

public class LinkedList<T> { ... }

LinkedList<?> l;

```

- But, we need to be careful about assigning values

```

public class LinkedList<T> { ... }

LinkedList<?> l = new LinkedList<String>();

l.add(new Object()); // Compile time error

```

- Compiler cannot guarantee the types match

Bounded Wildcards

- Suppose `Circle`, `Square` and `Rectangle` all extend `Shape`
- `Shape` has a method `draw()`
- All sub-classes override `draw()`
- We want a function to draw all elements in a list of `Shape` compatible objects

```

public static void drawAll(LinkedList<? extends Shape>) {
    Object o;
}

```

```

    Iterator i = l.get_iterator();

    while(i.has_next()) {
        o = i.get_next();
        o.draw();
    }
}

```

- Copying a `LinkedList`, using a wildcard

```

public static <? extends T, T> void listcopy(LinkedList<?> src, LinkedList<T> tgt) {
    Object o;
    Iterator i = src.get_iterator();

    while(i.has_next()) {
        o = i.get_next();
        tgt.add(o);
    }
}

```

- Can reverse the constraint, using `super`

```

public static <T, ? super T> void listcopy(LinkedList<T> src, LinkedList<?> tgt) {
    Object o;
    Iterator i = src.get_iterator();

    while(i.has_next()) {
        o = i.get_next();
        tgt.add(o);
    }
}

```

Summary

- Java generics are not covariant, unlike arrays
- Cannot substitute `Object` for `T` to get the most general type
- Instead, use type quantification `<T>` or wild card type variable `?`
- Wild card can be used wherever the type `T` is not required within the function
 - When `T` is not needed for return type, or to declare local variables
- Wild cards can be bounded
 - `LinkedList<? extends T>`
 - `LinkedList<? super T>`



Reflection

▼ Type	Lecture
📅 Date	@January 24, 2022
☰ Lecture #	4
🔗 Lecture URL	https://youtu.be/bPpWwY6YLyg
🔗 Notion URL	https://21f1003586.notion.site/Reflection-ec355c74651049079fb11673cd032bbc
# Week #	5

Reflection

According to Wikipedia

Reflective programming or **reflection** is the ability of a process to examine, introspect, and modify its own structure

- Two components involved in reflection
 - **Introspection**
A program can observe, and therefore reason about its own state
 - **Intercession**
A program can modify its execution state or alter its own interpretation or meaning

Reflection in Java

- Simple example of introspection

```
Employee e = new Manager(...);
...
if(e instanceof Manager) {
    ...
}
```

- What if we don't know the type that we want to check in advance?
- Suppose we want to write a function to check if two different objects are both instances of the same class?

```
public static boolean classequal(Object o1, Object o2) {
    ...
    // return true iff o1 and o2 point to objects of the same type
    ...
}
```

- Can't use `instanceof`
 - We will have to check across all defined classes
 - This is not even a fixed set
- Can't use generic type variables
 - The following code is syntactically disallowed

```
if (o1 instanceof T) { ... }
```

Introspection in Java

- We can extract the class of an object using `getClass()`
- Import package `java.lang.reflect`

```
import java.lang.reflect;

class MyReflectionClass {
    ...
    public static boolean classequal(Object o1, Object o2) {
        return o1.getClass() == o2.getClass();
    }
}
```

- What does `getClass()` return?
- An object of type `Class` that encodes class information

The class `Class`

- A version of `classequal` that explicitly uses this face

```
import java.lang.reflect.*;

class MyReflectionClass {
    ...
    public static boolean classequal(Object o1, Object o2) {
        Class c1, c2;
        c1 = o1.getClass();
        c2 = o2.getClass();

        return c1 == c2;
    }
}
```

- For each currently loaded class `c`, Java creates an object of type `Class` with information about `c`
- Encoding execution state as data — reification
 - Representing an abstract idea in a concrete form

Using the `Class` object

- We can create new instances of a class at runtime

```
...
Class c = obj.getClass();
Object o = c.newInstance();
// Create a new object of the same type as obj
...
```

- Can also get hold of the class object using the name of the class

```
...
String s = "Manager";
Class c = Class.forName(s);
Object o = c.newInstance();
...
```

- `...`, or, more compactly

```
...
Object o = Class.forName("Manager").newInstance();
```

The class `Class`

- From the `Class` object for class `c`, we can extract details about constructors, methods and fields of `c`
- Constructors, methods and fields themselves have structure
 - Constructors: arguments
 - Methods: arguments and return type
 - All three: modifiers `static`, `private` etc
- Additional classes `Constructor`, `Method`, `Field`
- Use `getConstructors()`, `getMethods()` and `getFields()` to obtain constructors, methods and fields of `c` in an array
- Extracting information about constructors, methods and fields

```
...
Class c = obj.getClass();
Constructor[] constructors = c.getConstructors();
Method[] methods = c.getMethods();
Field[] fields = c.getFields();
...
```

- `Constructor`, `Method`, `Field` in turn have functions to get further details

- Example → Get the list of parameters for each constructor

```
...
Class c = obj.getClass();
Constructor[] constructors = c.getConstructors();

for(int i = 0; i < constructors.length; ++i) {
    Class params[] = constructors[i].getParameterTypes();
    ...
}
```

- Each parameter list is a list of types
 - Return value is an array of type `Class[]`
- We can also invoke methods and examine/set values of fields

```
...
Class c = obj.getClass();
...
Method[] methods = c.getMethods();
Object[] args = { ... } // construct an array of arguments
methods[3].invoke(obj, args); // invoke methods[3] on obj with argument args
...
Field[] fields = c.getFields();
Object o = fields[2].get(obj); // get the value of fields[2] from obj
...
fields[3].set(obj, value); // set the value of fields[3] in obj to value
...
```

Reflection and Security

- Can we extract information about private methods, fields, ...?
- `getConstructors()`, ... only returns publicly defined values
- Separate functions to also include private components
 - `getDeclaredConstructors()`
 - `getDeclaredMethods()`
 - `getDeclaredFields()`
- Should this be allowed to all programs as this is a security issue

- Access to private components may be restricted through external security policies

Using reflection

- `BlueJ`, a programming environment to learn Java
- It can define and compile Java classes
- For compiled code, create object, invoke methods, examine state
- Uses reflective capabilities of Java — `BlueJ` need not internally maintain “debugging” information about each class

Limitations of Java reflection

- Cannot create or modify classes at run time
 - The following is not possible

```
Class c = new Class(...);
```
 - An environment like `BlueJ` must invoke Java compiler before you can use a new class
- Contrast with Python
 - `class XYZ`: can be executed at runtime in Python
- Other OO languages like Smalltalk allow redefining methods at run time



Java Generics at run time

Type	Lecture
Date	@January 24, 2022
Lecture #	5
Lecture URL	https://youtu.be/V7l0pVIIaEU
Notion URL	https://21f1003586.notion.site/Java-Generics-at-run-time-ab5e5c54b02c4f8cbaae7fddcae58f2e
Week #	5

Erasure of generic information

- Type erasure — Java does not keep record all versions of `LinkedList<T>` as separate types
 - Cannot write

```
if (s instanceof LinkedList<String>) { ... }
```
- At run time, all type variables are promoted to `Object`
 - `LinkedList<T>` becomes `LinkedList<Object>`
- Or, the upper bound, if one is available
 - `LinkedList<? extends Shape>` becomes `LinkedList<Shape>`
- Since no information about T is preserved, cannot use T in expressions like

```
if (o instanceof T) { ... }
```

Erasure and overloading

- Type erasure means the comparison in following code fragment returns `True`

```
o1 = new LinkedList<Employee>();
o2 = new LinkedList<Date>();

if(o1.getClass() == o2.getClass()) {
    // True, so the block is executed
}
```

- As a consequence, the following overloading is illegal

```
public class Example {
    public void printlist(LinkedList<String> strList) {}
}
```

```
public void printlist(LinkedList<Date> dateList) {}  
}
```

- Both functions have the same signature after type erasure

Arrays and Generics

- Recall the covariance problem for arrays
 - If `S extends T` then `S[] extends T[]`
- Can lead to run time type errors

```
ETicket[] elecarr = new ETicket[10];  
Ticket[] ticketarr = elecarr; // OK. ETicket[] is a subtype of Ticket[]  
...  
ticketarr[5] = new Ticket(); // Not OK. ticketarr[5] refers to an ETicket
```

- To avoid similar problems, can declare a generic array, but cannot instantiate it

```
T[] newarray; // OK  
newarray = new T[100]; // Cannot create
```

- An ugly workaround that generates compiler warning, but works

```
T[] newarray;  
newarray = (T[]) new Object[100];
```

Wrapper classes

- Type erasure — at run time, all type variables are promoted to `Object`
 - `LinkedList<T>` becomes `LinkedList<Object>`
- Basic types `int`, `float`, ... are not compatible with `Object`
- We cannot use basic type in place of a generic type variable `T`
 - We cannot instantiate `LinkedList<T>` as `LinkedList<int>`, `LinkedList<double>`, ...
- Wrapper class for each basic type:

Basic type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long

Basic type	Wrapper Class
float	Float
double	Double
boolean	Boolean
char	Character

- All wrapper classes other than `Boolean`, `Character` extend the class `Number`

- Converting from basic type to wrapper class and back

```
int x = 5;  
Integer myx = Integer(x);  
int y = myx.intValue();
```

- Similarly, `byteValue()`, `doubleValue()`, ...

- Autoboxing — implicit conversion between base types and their wrapper types

```
int x = 5;  
Integer myx = x;  
int y = myx;
```

- Use wrapper types in generic data structures

Summary

- Java generics come with some restrictions
- Information about type variables is erased at runtime
 - `LinkedList<T>` becomes `LinkedList<Object>`
 - `LinkedList<? extends Shape>` becomes `LinkedList<Shape>`
- Limits the use reflection on generic types — ***cannot write***
 - `if (o instanceof LinkedList<String>) { ... }`
 - `if (o instanceof T) { ... }`
- Cannot overload function signatures using instantiation of generic types
- Cannot instantiate arrays of generic type
- Need to box built-in types using wrapper types